

OLD DOMINION
UNIVERSITY

CCL Group Training

OpenFOAM Tutorial: aachenBomb

Date 10/02/2026

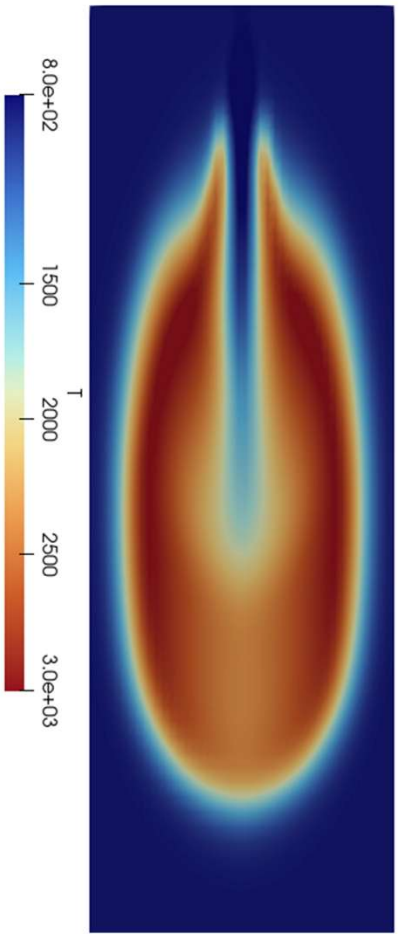
OpenFOAM Tutorial: aachenBomb

- **Physical Problem:** 3D transient, constant-volume spray combustion and auto-ignition of a fuel spray in a closed vessel (bomb).

- **Solver:** sprayFoam (transient solver for compressible, turbulent, reacting flows with Lagrangian spray particle tracking).

- **Directory Structure:** Standard OpenFOAM setup (0/, constant/, system/).

- **Core Physical Phenomena:**
  - **Liquid Phase Injection:** High-velocity fuel injection through a fine nozzle into a quiescent, high-temperature, high-pressure ambient gas (N_2 / O_2 mixture).
  - **Liquid Breakup & Evaporation:** Primary and secondary droplet breakup due to aerodynamic shear, followed by phase change and vapor mixing.
  - **Auto-Ignition:** Gas-phase chemistry leading to low- and high-temperature ignition reactions.
  - **Flame Propagation & Pressure Rise:** Rapid heat release driving a transient pressure spike inside the vessel.



OpenFOAM Tutorial: aachenBomb

- **Computational Domain & Grid Strategy:**
  - **Geometry:** A 2D/3D wedge/sector mesh or a 3D rectangular/axisymmetric block taking advantage of wedge or symmetry boundary conditions to reduce cell count.
  - **Liquid Breakup & Evaporation:** Primary and secondary droplet breakup due to aerodynamic shear, followed by phase change and vapor mixing.
  - **Auto-Ignition:** Gas-phase chemistry leading to low- and high-temperature ignition reactions.
  - **Flame Propagation & Pressure Rise:** Rapid heat release driving a transient pressure spike inside the vessel.

- **Solver Architecture:**
  - **Lagrangian-Eulerian (LE) Framework:**
    - **Eulerian Gas Phase:** Solves Navier-Stokes, species transport, energy and turbulence equations on the fixed mesh.
    - **Lagrangian Particle Tracking (LPT):** Tracks individual droplet computational parcels through the Eulerian flow field.
  - **Phase Coupling:** Two-way coupling exchanges mass (evaporation), momentum (drag), and energy (heat transfer) between droplets and the continuous gas phase.

OpenFOAM Tutorial: aachenBomb

- **Key Case Files & Directory Layout:**
  - 0/ Directory — Initial & Boundary Conditions
  - constant/ — Physical, Chemical & Multi-Phase Models
  - system/ — Mesh, Numerics & Simulation Control

```
aachenBomb/
├── 0/                          <-- Initial & Boundary Conditions
│   ├── T, p, U                 <-- Thermodynamic & flow fields
│   ├── k, epsilon / omega      <-- Turbulence fields
│   └── Ydefault, N2, O2, C7H16 <-- Mass fractions of chemical species
├── constant/                   <-- Physical & Model Properties
│   ├── thermophysicalProperties <-- Thermodynamics & transport definition
│   ├── chemistryProperties     <-- Chemistry ODE solver configuration
│   ├── reactions               <-- Chemical kinetic mechanism
│   ├── combustionProperties    <-- Combustion model (e.g., PaSR)
│   ├── sprayCloudProperties    <-- Injection, breakup, drag, & evaporation
│   └── momentumTransport       <-- RANS turbulence model (e.g., k-epsilon)
└── system/                     <-- Numerical & Time Controls
    ├── blockMeshDict           <-- Mesh generation instructions
    ├── controlDict             <-- Timestep, output frequency, maxCo
    ├── fvSchemes               <-- Spatial/temporal discretization choices
    └── fvSolution              <-- Linear solvers, tolerances, PIMPLE loops
```

OpenFOAM Tutorial: aachenBomb

0/ Directory — Initial & Boundary Conditions:
This directory contains field files representing state variables at time $t = 0$. Each file defines internal field values across all mesh cells and boundary conditions applied to domain faces.

- Primary Dynamic Fields (T, p, U):
  - p / T: Sets high initial ambient pressure and temperature inside the closed vessel required to reach auto-ignition thresholds.
  - U: Velocity field, typically set to zero (uniform (0 0 0)) for quiescent baseline conditions before injection starts.

- Turbulence Quantities (k, epsilon, omega):
  - Defines initial turbulent kinetic energy (k) and dissipation rate (ϵ or ω) to initialize the RANS model.

- Chemical Species Mass Fractions (Ydefault, N2, O2, C7H16):
  - Tracks mass fraction fields Y_i for each species in the mechanism.
  - N2, O2: Defines background ambient gas composition (air mixture).
  - C7H16: Fuel species field, initialized to zero in the gas phase before spray injection.
  - Ydefault: Fallback boundary condition template for secondary chemical species created during reactions.

OpenFOAM Tutorial: aachenBomb

0/p

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  v2606                                 |
|   \\  /    A nd           | Website:  www.openfoam.com                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       volScalarField;
    object      p;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

dimensions          [1 -1 -2 0 0 0 0];

internalField       uniform 5e06;

boundaryField
{
    walls
    {
        type            zeroGradient;
    }
}


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
```

0/U

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  v2606                                 |
|   \\  /    A nd           | Website:  www.openfoam.com                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       volVectorField;
    object      U;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

dimensions          [0 1 -1 0 0 0 0];

internalField       uniform (0 0 0);

boundaryField
{
    walls
    {
        type                fixedValue;
        value               uniform (0 0 0);
    }
}


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
```

OpenFOAM Tutorial: aachenBomb

constant/ Directory — Physical, Chemical & Multi-Phase Models:
This directory governs all physical laws, sub-grid scale closures, thermophysical modeling, and Lagrangian particle behavior.

- thermophysicalProperties: Defines fluid thermophysical properties, state equations (e.g., perfectGas), mixing rules, and transport models (e.g., sutherland).

- chemistryProperties: Controls chemical kinetics ODE solvers (e.g., seulex, EulerSI), solver tolerances, and chemistry switch (chemistry on/off).

- combustionProperties: Configures the turbulence-chemistry interaction (TCI) model (e.g., PaSR - Partially Stirred Reactor or laminar).

- sprayCloudProperties: Primary dictionary for Lagrangian Spray Tracking:
  - **Injection**: Injector location, hole size, injection profile, mass flow rate, and parcel count.
  - **Breakup Models**: Primary/secondary breakup parameters (e.g., TAB or ReitzKHRT).
  - **Evaporation & Drag**: Liquid property parameters, phase change, heat transfer, and sphere drag coefficients.

OpenFOAM Tutorial: aachenBomb

constant/chemistryProperties

```
/*---------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  v2606                                 |
|   \\  /    A nd           | Website:  www.openfoam.com                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      chemistryProperties;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

chemistryType
{
    solver            ode;
}

chemistry       on;

initialChemicalTimeStep 1e-07;

odeCoeffs
{
    solver          seulex;
    eps             0.05;
}


// ************************************************************************* //
```

constant/sprayCloudProperties

```
/*---------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  v2606                                 |
|   \\  /    A nd           | Website:  www.openfoam.com                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      sprayCloudProperties;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

solution
{
    active          true;
    coupled         true;
    transient       yes;
    cellValueSourceCorrection on;
    maxCo           0.3;

    sourceTerms
    {
        schemes
        {
            rho             explicit 1;
            U               explicit 1;
            Yi              explicit 1;
            h               explicit 1;
            radiation       explicit 1;
        }
    }
```

OpenFOAM Tutorial: aachenBomb

system/ Directory — Mesh, Numerics & Simulation Control:
This directory specifies how OpenFOAM discretizes and numerically solves the governing partial differential equations (PDEs).

- blockMeshDict: Native parametric mesh generator script defining domain vertices, block connectivity, cell count distributions, grading ratios, and patch boundary types.

- controlDict: Master runtime controller defining start/stop times (startTime, endTime), initial timestep size (deltaT), adaptive time-stepping (adjustTimeStep), target Courant numbers (maxCo, maxDi), and write control settings.

- decomposeParDict: Configures domain decomposition strategies (e.g., simple, hierarchical, or scotch) and process counts (numberOfSubdomains) for running the simulation in parallel across multiple CPU cores.

- fvSchemes: Specifies numerical discretization schemes for differential operators:
  - ddtSchemes: Time integration (e.g., 1st-order implicit Euler or 2nd-order CrankNicolson).
  - gradSchemes, divSchemes, laplacianSchemes: Spatial gradient, convection, and diffusion operators (e.g., bounded TVD/NVD schemes for species and energy).

OpenFOAM Tutorial: aachenBomb

system/blockMeshDict

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  v2606                                 |
|   \\  /    A nd           | Website:  www.openfoam.com                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      blockMeshDict;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

scale   0.001;

vertices
(
    (-10 0 -10)
    (-10 0 10)
    (10 0 10)
    (10 0 -10)
    (-10 100 -10)
    (-10 100 10)
    (10 100 10)
    (10 100 -10)
);
```

constant/decomposeParDict

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  v2606                                 |
|   \\  /    A nd           | Website:  www.openfoam.com                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      decomposeParDict;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

numberOfSubdomains 8;

method          simple;

coeffs
{
    n           (1 8 1);
}

// ************************************************************************* //
```

OpenFOAM Tutorial: aachenBomb

system/blockMeshDict

```
scale   0.001;

vertices
(
    (-10 0 -10)
    (-10 0 10)
    (10 0 10)
    (10 0 -10)
    (-10 100 -10)
    (-10 100 10)
    (10 100 10)
    (10 100 -10)
);
```

Vertex ID #0

Vertex ID #7

- OpenFOAM operates internally in SI units (meters).
- Setting scale 0.001; acts as a global multiplier, converting all vertex coordinates from millimeters (mm) to meters (m) (e.g., 10 mm → 0.01 m).

Geometry & Coordinate Definition (vertices)

- Defines the 8 corner points of the 3D bounding box in Cartesian coordinates (x, y, z)
- Vertex IDs assigned (0 to 7)

Block Connectivity & Mesh Resolution (blocks)

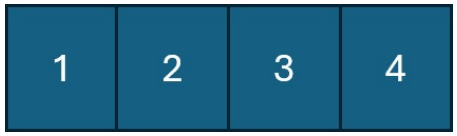
```
blocks
(
    hex (0 1 2 3 4 5 6 7) (41 41 100) simpleGrading (1 1 1)
);
```

- Defines a single 8-node hexahedral block connecting vertices 0 through 7.
- Specifies the cell counts along each local axis (N_x, N_y, N_z)
- Uniform cell distribution along all three axes (no mesh stretching)

OpenFOAM parallel computing

OpenFOAM uses domain decomposition via the Message Passing Interface (MPI) to handle parallel computing across multiple CPU cores or clusters.

1. **Domain Decomposition:** Before running a simulation in parallel, the computational mesh and initial fields are split into subdomains—one for each MPI process/core.
   - Tool: Executed using the `decomposePar` utility.
   - Configuration: Defined in `system/decomposeParDict`
   - Common Methods:
     - `simple` / `hierarchical`: Splits the domain along Cartesian axes x, y, z.
     - `scotcha` / `ptscotch`: Uses graph partitioning to minimize the surface area of boundaries between subdomains, reducing network communication overhead (recommended for complex geometries).

   After running `decomposePar`, OpenFOAM creates `processor0`, `processor1`, ..., `processorN` directories inside the case directory, each containing its piece of the mesh and field data.

2. **Communication via Processor Boundaries:** When a domain is split, OpenFOAM creates a special patch type called `processor` at the interfaces between neighboring subdomains.

OpenFOAM parallel computing

Here is a step-by-step guide to setting up and running sprayFoam in parallel across multiple CPU cores.

1. **Configure system/decomposeParDict**
   To run in parallel, OpenFOAM needs instructions on how to split the mesh and field data among CPU cores. Configure the fila named decomposeParDict in your system/ directory:

```
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      decomposeParDict;
}
// * * * * * * * * * * * * * * * * *

numberOfSubdomains 8;

method          simple;

coeffs
{
    n           (1 8 1);
}
```

- numberOfSubdomains: The total number of CPU cores/processes you want to allocate.
- method simple;: Automatically split a mesh domain geometrically along the Cartesian axes (x, y, z) based on the vector provided in the coeffs dictionary.
- coeffs { n (1 8 1); } instructs the algorithm to split the domain:
  - 1 slice along the x-axis
  - 8 slice along the y-axis
  - 1 slice along the z-axis

OpenFOAM parallel computing

2. **Generate Mesh and Initialize Fields:**
   Before decomposing, ensure the single-processor mesh and initial conditions (`0/` directory) are generated and configured.

   ```
   $ blockMesh
   ```

3. **Decompose the Domain:**
   Split the mesh and all field files in `0/` into separate subdomains for each core:

   ```
   $ decomposePar
   ```

   `decomposePar` utility creates `processor0`, `processor1`, ..., `processorN-1` directories inside your case directory. Each folder contains its portion of the mesh, boundary patches, initial conditions, and Lagrangian spray cloud initializations.

4. **Run sprayFoam in Parallel with MPI:**

   ```
   $ mpirun –np 8 sprayFoam –parallel | tee log.sprayFoam
   ```

OpenFOAM parallel computing

5. **Reconstruct the Parallel Domain for Post-Processing:**
   Once the simulation completes (or reaches a saved timestep), stitch the divided processor* output directories back into a single global dataset for visualization:

   ```
   $ reconstructPar
   ```

   reconstructPar reads the timestep results inside each processor* folder and reconstructs complete field data in the top-level case directory (e.g., 0.001/, 0.002/).

6. **Visualize Results in ParaView:**
   Create a .foam dummy file if it doesn't exist:

   ```
   $ touch aachemBomb.foam
   ```

   Open it in ParaView.

Workflow Quick Reference Commands

```
blockMesh
decomposePar
mpirun -np 4 sprayFoam -parallel | tee log.sprayFoam
reconstructPar
```